



Exercise 2: Process Synchronization Implementation

Lappeenranta–Lahti University of Technology LUT

Computational Science and Artificial Intelligence

CT30A3350 Operating Systems and System Programming (Lahti)

Date: 15.04.2026

Md Hamidur Rahman Khan, 002475552

Examiner: Ankit Agrawal, PhD

Task 1

The program used three semaphores. ‘mutex’ provided mutual exclusion when accessing the shared buffer. ‘empty’ kept track of how many buffer slots were still empty, while ‘full’ kept track of how many were already occupied. A producer had to wait until an empty slot was available before inserting an item, and a consumer had to wait until there was an item in the buffer before removing one.

```
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex2/task1$ gcc -o producer_consumer producer_consumer.c -pthread -lrt
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex2/task1$ ./producer_consumer
Producer 1 produced 101 at index 0
Producer 2 produced 201 at index 1
Consumer 1 consumed 101 from index 0
Consumer 2 consumed 201 from index 1
Producer 1 produced 102 at index 2
Consumer 2 consumed 102 from index 2
Producer 2 produced 202 at index 3
Consumer 1 consumed 202 from index 3
Producer 2 produced 203 at index 4
Consumer 2 consumed 203 from index 4
Producer 1 produced 103 at index 0
Consumer 1 consumed 103 from index 0
Producer 2 produced 204 at index 1
Producer 1 produced 104 at index 2
Consumer 2 consumed 204 from index 1
Consumer 1 consumed 104 from index 2
Producer 2 produced 205 at index 3
Consumer 2 consumed 205 from index 3
Producer 1 produced 105 at index 4
Consumer 1 consumed 105 from index 4
Parent inserted termination item -1 at index 0
Parent inserted termination item -1 at index 1
Consumer 2 consumed -1 from index 0
Consumer 1 consumed -1 from index 1
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex2/task1$
```

Figure 1. Terminal showing compile and run command with resulting output of the producer consumer problem

The output showed that producers and consumers ran concurrently. Producers added items to the shared buffer, while consumers removed them in a properly synchronized manner. Although the exact execution order changed between runs, the synchronization rules were followed every time.

The ‘empty’ semaphore prevented producers from adding items when the buffer was full. The ‘full’ semaphore prevented consumers from removing items when the buffer was empty. The ‘mutex’ semaphore ensured that only one process could access the shared buffer at a time, which prevented race conditions on the buffer and index variables. As a result, the bounded buffer problem was handled correctly.

Task 2

Five chopstick semaphores were used, and each initialized to 1. So only one philosopher could hold a chopstick at a time. An additional semaphore, 'room', was initialized to 4, so a maximum of four philosophers could try to pick up chopsticks at the same time. This eventually prevented circular waiting and avoided deadlock.

In Figure 2.1 it can be seen that all the philosophers took turns between thinking and eating. Each philosopher picked up both the left and right chopsticks before eating and released the chopsticks after eating. The program finished successfully each time.

The chopstick semaphores guaranteed mutual exclusion for each chopstick. The 'room' semaphore prevented all five philosophers from trying to pick up chopsticks at the same time, which removed the deadlock condition. For that the simulation ran correctly without any philosopher getting stuck holding one chopstick while waiting indefinitely for the other.

```
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex2/task2$ gcc -o dining_philosophers dining_philosophers.c -pthread -lrt
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex2/task2$ ./dining_philosophers
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 3 is thinking
Philosopher 2 is thinking
Philosopher 4 is thinking
Philosopher 0 picked up left chopstick 0
Philosopher 0 picked up right chopstick 1
Philosopher 0 is eating (round 1)
Philosopher 3 picked up left chopstick 3
Philosopher 3 picked up right chopstick 4
Philosopher 3 is eating (round 1)
Philosopher 2 picked up left chopstick 2
Philosopher 0 finished eating and released chopsticks
Philosopher 0 is thinking
Philosopher 1 picked up left chopstick 1
Philosopher 3 finished eating and released chopsticks
Philosopher 3 is thinking
Philosopher 2 picked up right chopstick 3
Philosopher 4 picked up left chopstick 4
Philosopher 2 is eating (round 1)
Philosopher 4 picked up right chopstick 0
Philosopher 4 is eating (round 1)
Philosopher 4 finished eating and released chopsticks
Philosopher 4 is thinking
Philosopher 0 picked up left chopstick 0
Philosopher 2 finished eating and released chopsticks
Philosopher 2 is thinking
Philosopher 3 picked up left chopstick 3
Philosopher 3 picked up right chopstick 4
Philosopher 3 is eating (round 2)
Philosopher 1 picked up right chopstick 2
Philosopher 1 is eating (round 1)
Philosopher 3 finished eating and released chopsticks
Philosopher 3 is thinking
Philosopher 4 picked up left chopstick 4
Philosopher 1 finished eating and released chopsticks
```

Figure 2.1. Terminal showing compile and run command with part 1 of the resulting output of the dining philosophers problem

```

Philosopher 1 picked up right chopstick 2
Philosopher 1 is eating (round 1)
Philosopher 3 finished eating and released chopsticks
Philosopher 3 is thinking
Philosopher 4 picked up left chopstick 4
Philosopher 1 finished eating and released chopsticks
Philosopher 1 is thinking
Philosopher 2 picked up left chopstick 2
Philosopher 2 picked up right chopstick 3
Philosopher 2 is eating (round 2)
Philosopher 0 picked up right chopstick 1
Philosopher 0 is eating (round 2)
Philosopher 0 finished eating and released chopsticks
Philosopher 0 is thinking
Philosopher 4 picked up right chopstick 0
Philosopher 4 is eating (round 2)
Philosopher 1 picked up left chopstick 1
Philosopher 2 finished eating and released chopsticks
Philosopher 2 is thinking
Philosopher 3 picked up left chopstick 3
Philosopher 1 picked up right chopstick 2
Philosopher 1 is eating (round 2)
Philosopher 1 finished eating and released chopsticks
Philosopher 4 finished eating and released chopsticks
Philosopher 1 is thinking
Philosopher 4 is thinking
Philosopher 2 picked up left chopstick 2
Philosopher 0 picked up left chopstick 0
Philosopher 0 picked up right chopstick 1
Philosopher 0 is eating (round 3)
Philosopher 3 picked up right chopstick 4
Philosopher 3 is eating (round 3)
Philosopher 3 finished eating and released chopsticks
Philosopher 2 picked up right chopstick 3
Philosopher 2 is eating (round 3)
Philosopher 4 picked up left chopstick 4
Philosopher 0 finished eating and released chopsticks
Philosopher 1 picked up left chopstick 1
Philosopher 4 picked up right chopstick 0

```

Figure 2.2. Terminal showing part 2 of the resulting output of the dining-philosophers problem

```

Philosopher 2 is eating (round 3)
Philosopher 4 picked up left chopstick 4
Philosopher 0 finished eating and released chopsticks
Philosopher 1 picked up left chopstick 1
Philosopher 4 picked up right chopstick 0
Philosopher 4 is eating (round 3)
Philosopher 2 finished eating and released chopsticks
Philosopher 1 picked up right chopstick 2
Philosopher 1 is eating (round 3)
Philosopher 4 finished eating and released chopsticks
Philosopher 1 finished eating and released chopsticks
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex2/task2$

```

Figure 2.3. Terminal showing part 3 of the resulting output of the dining-philosophers problem